

KamiLimu • Cohort 10 • Cloud Engineering Track

Session 1

Compute Services & Storage Solutions

A beginner's foundation in cloud computing, virtualization, compute, and storage on AWS — with self-hosted alternatives.

PRE-SESSION STUDY MATERIAL

Read before tomorrow's session. Highlight anything unclear — we'll cover it live.

Prepared by: Mark Murithi Kibara

Estimated reading time: 35–45 minutes

What's in this guide

This document is your warm-up for our first cloud engineering session. It covers the big ideas, the AWS services we'll touch, and self-hosted alternatives so you can build intuition before we go hands-on.

1. Why cloud, and what makes it different 3
2. The service models: IaaS, PaaS, SaaS 5
3. Deployment models: public, private, hybrid 6
4. Virtualization — the magic behind compute 7
5. Regions, Availability Zones, and physical reality 8
6. Compute on AWS: EC2, ECS, EKS 9
7. Containers vs. virtual machines 12
8. Storage on AWS: EBS, EFS, S3 13
9. Object storage deep-dive + Minio 16
10. The Shared Responsibility Model 17
11. Cost awareness for beginners 18
12. Pre-session checklist 19
13. Glossary of terms 20
14. Further reading 22

1. Why cloud, and what makes it different

Before we touch AWS, let's answer a question many people skip: what actually makes "the cloud" different from a regular computer or a server in your office?

A useful (but incomplete) one-liner is: the cloud is someone else's computer. That's technically true, but it misses the point. What matters is the operating model — how you get, pay for, and manage that computer.

The five characteristics that define cloud computing

The U.S. National Institute of Standards and Technology (NIST) defines cloud computing using five essential characteristics. You'll hear these repeatedly throughout the cohort:

- **On-demand self-service:** You can provision a server, a database, or a storage bucket in minutes by clicking a button or running a command. No ticket. No procurement. No human in the loop.
- **Broad network access:** Resources are accessible over the internet from any device — laptops, phones, IoT sensors.
- **Resource pooling:** The provider's hardware serves many customers at once using virtualization. You don't know (or care) which physical server is running your workload.
- **Rapid elasticity:** Capacity can expand or shrink quickly — sometimes automatically — to match demand.
- **Measured service:** Usage is metered (CPU-seconds, GB-months, requests). You pay only for what you use.

From CapEx to OpEx

Traditional IT spending was capital expenditure (CapEx): you buy a server for KES 800,000, depreciate it over five years, and hope you sized it correctly. Cloud spending is operational expenditure (OpEx): you pay monthly for what you actually used, like an electricity bill.

This shift matters because it changes who can build what. A student in Nairobi can now spin up the same caliber of infrastructure that, ten years ago, was reserved for companies with seven-figure IT budgets.

Scalability vs. elasticity — these are NOT the same

Beginners often use these words interchangeably. They are related but distinct:

- **Scalability** is the ability to handle increased load — by either adding more machines (horizontal) or making a machine bigger (vertical). It's a capability.

- **Elasticity** is the ability to automatically scale up AND scale down based on demand. It's a behavior — and it's what saves you money.

Analogy

Scalability is owning a stretchy waistband. Elasticity is the waistband stretching and shrinking on its own as you eat lunch and then digest it. Cloud done well is elastic — your bill should shrink when traffic dies down.

The trade-offs — cloud is not a silver bullet

It's worth being honest about the downsides. Knowing them makes you a better engineer:

- Costs can spiral if resources are left running — "bill shock" is real, and it's how most beginners get burned.
- You depend on the provider's availability. If AWS us-east-1 has a bad day, half the internet has a bad day.
- Lock-in is a risk. Heavy use of proprietary services makes it hard to move to another cloud later.
- Data sovereignty: some workloads have legal restrictions on where data can live. This is partly why self-hosted options like Minio exist.

2. The service models: IaaS, PaaS, SaaS

Cloud services come in three classic layers. The difference is how much you manage versus how much the provider manages.

Layer	IaaS — Infrastructure	PaaS — Platform	SaaS — Software
You manage	OS, runtime, app, data	App and data only	Nothing — just use it
Provider manages	Hardware, virtualization, networking	Everything below the runtime	Everything
AWS example	EC2, EBS, VPC	Elastic Beanstalk, App Runner, RDS	Amazon Chime, WorkMail
Everyday example	Renting an empty plot of land	Renting a serviced apartment	Booking an Airbnb stay
Trade-off	Maximum control, maximum effort	Less control, faster delivery	Zero control, fastest delivery

Where this cohort focuses

Most of our work will be at the IaaS layer with some PaaS sprinkled in. Why? Because IaaS forces you to understand the building blocks. Once you understand EC2, you'll understand why Elastic Beanstalk exists. Going the other way is harder.

There's also a less formal fourth model worth knowing:

- **FaaS (Function as a Service)** / Serverless: You upload code; the provider runs it in response to events. You don't manage servers at all, and you pay only when your code actually executes. AWS Lambda is the canonical example — and it's on the curriculum for a later session.

3. Deployment models: public, private, hybrid, multi-cloud

Service models (IaaS/PaaS/SaaS) describe what you're getting. Deployment models describe where it lives and who owns it.

Public cloud

Infrastructure owned and operated by a third-party provider (AWS, Azure, Google Cloud), shared across many customers. This is what most people mean when they say "cloud."

Private cloud

Cloud infrastructure dedicated to a single organization. It can be on-premises (in your own data center) or hosted by a provider. Common in banking, government, and healthcare where data residency or compliance matters.

Hybrid cloud

A mix of public and private cloud connected so that workloads and data can move between them. A common pattern: keep sensitive data on-prem, burst to public cloud for compute-heavy work.

Multi-cloud

Using more than one public cloud provider — for example, AWS for compute and Google Cloud for AI/ML. Reduces lock-in but increases complexity. Our cohort covers this in a later session.

Self-hosting fits here

When we self-host Minio on EC2 in our demo, we're blending models: we're using a public-cloud VM (IaaS) to run a private object store. That's a hybrid mindset on day one.

4. Virtualization — the magic behind compute

Before you can understand EC2, you need to understand virtualization. Without it, the cloud as we know it would not exist.

The core idea

A physical server is powerful — often with 64+ CPU cores and hundreds of gigabytes of RAM. Most workloads only need a small slice of that. Virtualization slices one physical machine into many isolated "virtual machines" (VMs), each behaving like a complete computer with its own operating system.

The hypervisor

A hypervisor is the software (or firmware) layer that creates and manages VMs. It sits between the physical hardware and the guest operating systems, allocating CPU, memory, and I/O across them while keeping each VM isolated from the others.

Two types:

- **Type 1 (bare-metal):** Runs directly on hardware. Examples: VMware ESXi, Microsoft Hyper-V, Xen, KVM. This is what cloud providers use.
- **Type 2 (hosted):** Runs on top of a regular OS. Examples: VirtualBox, VMware Workstation. Good for learning on your laptop.

What's running your EC2 instance?

AWS uses a custom hypervisor called Nitro (replacing the earlier Xen-based system). When you launch a t3.micro EC2 instance, you're getting a small slice of a much bigger physical "Nitro" host machine, isolated from whoever else is running on that same box.

Why this matters for you

Understanding virtualization explains a few things that often confuse beginners:

- Why an "EC2 instance" can be created and destroyed in seconds — there's no physical hardware being installed.
- Why you can resize an instance (give it more CPU/RAM) by just restarting it — the hypervisor just allocates a different slice.
- Why "noisy neighbors" can sometimes be a problem on cheap shared instance types — you're sharing the underlying box.
- Why containers (which we'll cover next) were such a big deal — they're a lighter, faster alternative to spinning up a whole VM.

5. Regions, Availability Zones, and physical reality

Despite the name, the cloud is very physical. AWS owns or leases huge data centers around the world. How those data centers are organized affects your design choices — and, by extension, whether your application survives the next power outage.

Regions

A Region is a geographic area where AWS clusters data centers. Each Region has a name like us-east-1 (Northern Virginia), eu-west-1 (Ireland), af-south-1 (Cape Town), or me-south-1 (Bahrain).

Why Regions matter:

- Latency — choose a Region close to your users.
- Data residency — some laws require data to stay in a specific country.
- Pricing — the same service costs different amounts in different Regions.
- Service availability — not every AWS service exists in every Region.

Availability Zones (AZs)

Each Region is made up of multiple Availability Zones — typically three or more. An AZ is one or more discrete data centers with redundant power, networking, and connectivity. AZs in a Region are physically separated (often tens of kilometres apart) but connected by high-speed, low-latency private links.

Why this matters in plain English

If a power outage, a fire, or a flood takes down one data center (an AZ), your application can keep running because it has copies in another AZ in the same Region. This is the foundation of "high availability" on the cloud. Production workloads always span at least two AZs.

Edge locations

AWS also has hundreds of Edge Locations — smaller sites used to cache content closer to end users. Services like CloudFront (the AWS CDN) and Route 53 (DNS) live here. We'll touch on these in the networking session.

6. Compute on AWS: EC2, ECS, EKS

"Compute" is the broad term for anything that runs your code. AWS has many compute services, but for this session we focus on three: EC2 (virtual machines), ECS (managed containers), and EKS (managed Kubernetes).

EC2 — Elastic Compute Cloud

EC2 gives you virtual machines on demand. It's the original AWS service (launched in 2006) and still the most-used. When someone says "spin up a server on AWS," they almost always mean EC2.

Key EC2 concepts you'll meet in the demo

- **Instance:** A single running virtual machine.
- **AMI (Amazon Machine Image):** The template a new instance is built from — like a snapshot of an OS plus pre-installed software. There are AMIs for Ubuntu, Amazon Linux, Windows Server, and many custom appliances.
- **Instance type:** The "size" of the VM — e.g., t3.micro, m5.large, c6g.xlarge. The letter family hints at the workload (t = burstable, m = balanced, c = compute-optimized, r = memory-optimized, g = GPU). The number is the generation. The suffix is the size.
- **Key pair:** A public/private SSH key pair. You'll need the private key to log in to your instance. Lose it and you cannot recover access (treat it like a password).
- **Security group:** A virtual firewall attached to your instance. Controls which ports are open to which IP addresses.
- **User data:** A script that runs the first time an instance boots — handy for installing software automatically (we'll use this for the Minio demo).

EC2 purchase options — why your bill depends on this

Option	How it works	Best for
On-Demand	Pay per second / hour with no commitment.	Short, unpredictable workloads — and learning.
Reserved	Commit to 1 or 3 years for up to ~72% discount.	Steady-state workloads you know you'll run.
Savings Plans	Commit to a dollar-amount per hour, get a discount; flexible across instance types.	Predictable spend but variable workload mix.

Option	How it works	Best for
Spot	Bid on unused capacity — up to ~90% off. AWS can take it back with 2 minutes' notice.	Fault-tolerant batch jobs, CI runners, ML training.
Dedicated Host	Physical server reserved for you alone.	Licensing or compliance requirements.

ECS — Elastic Container Service

ECS is AWS's own managed container orchestrator. You give it Docker images and tell it how many copies you want; ECS handles scheduling, scaling, and health checks.

ECS has two "launch types":

- **EC2 launch type:** You manage a fleet of EC2 instances; ECS places containers on them. Cheaper but more work.
- **Fargate launch type:** Serverless containers — AWS runs the underlying infrastructure for you. You only see the containers. Easier but more expensive per hour.

EKS — Elastic Kubernetes Service

EKS is managed Kubernetes. Kubernetes is the open-source container orchestrator that has become the industry standard. EKS hands you a Kubernetes control plane that AWS keeps running, and you bring the worker nodes (or use Fargate).

Why both ECS and EKS exist:

- ECS is simpler and cheaper to operate, but it's AWS-only.
- EKS gives you the full Kubernetes ecosystem — portable across clouds, much larger community, much steeper learning curve.

Mentor's tip

Don't reach for Kubernetes on day one. For a small app, EC2 with Docker — or even Elastic Beanstalk — is faster, cheaper, and easier to debug. EKS shines when you have many services, many teams, and serious scale.

The compute spectrum on AWS

To put it all in one picture, here's the spectrum from "most control" to "most managed":

EC2	ECS / EKS	Fargate	Lambda
Virtual machines you fully manage	Container orchestration on your VMs	Serverless containers	Serverless functions
Maximum control	→	→	Maximum abstraction

7. Containers vs. virtual machines

This is one of the most important mental models in modern infrastructure. Beginners often think containers and VMs are the same thing, but they're fundamentally different.

Virtual machines

A VM is a complete, virtualized computer. It has its own operating system kernel, its own file system, and its own network stack — all running on top of a hypervisor. Boot time: tens of seconds to minutes. Size: gigabytes.

Containers

A container is a lightweight, isolated package of an application and its dependencies. Crucially, all containers on a host share the host's OS kernel — they don't bring their own. Boot time: milliseconds. Size: megabytes.

House vs. apartment analogy

A VM is a standalone house — own foundation, own plumbing, own electrical wiring. A container is an apartment in a shared building — own furniture and decoration, but shared structural systems. The apartment is faster to build, cheaper to maintain, and easier to replicate, but you're trusting the building (the host OS) to stay solid.

Aspect	Virtual Machine	Container
OS	Full guest OS per VM	Shares host kernel
Size	GBs	MBs
Boot time	Seconds to minutes	Milliseconds
Isolation	Stronger (hardware-level)	Weaker (OS-level)
Density	Tens per host	Hundreds to thousands per host
Portability	OS-bound	Runs anywhere Docker runs
Use case	Long-running, OS-specific workloads	Microservices, CI/CD, scalable apps

In practice you'll often combine them: containers running on top of VMs. That's exactly what ECS-on-EC2 and EKS-on-EC2 do.

8. Storage on AWS: EBS, EFS, S3

Storage on the cloud comes in three flavors. The right choice depends on what you're storing and how it will be accessed.

Aspect	Block (EBS)	File (EFS)	Object (S3)
Mental model	A raw hard drive you plug into one server	A shared network drive in an office	A giant warehouse of labeled boxes
Access	Attached to one EC2 instance at a time	Mounted by many EC2 instances simultaneously	HTTP(S) API from anywhere
Structure	Blocks (low level, like a disk)	Folders and files (POSIX)	Flat — every object has a unique key
Performance	Very high IOPS, very low latency	Good for shared workloads, slower than EBS	High throughput, higher latency
Scalability	Up to 64 TiB per volume	Petabytes, grows automatically	Effectively unlimited
Typical use	Databases, OS disk, transactional apps	Shared content for web servers, CMS, dev environments	Backups, static websites, logs, media, data lakes
Pricing	Per GB-month provisioned (paid even when empty)	Per GB-month used (no provisioning)	Cheapest per GB; small fee per request

EBS — Elastic Block Store

EBS volumes look like a raw disk to your EC2 instance. You partition them, format them, mount them, and use them just like a physical hard drive. They live in a single Availability Zone, so for high availability you need to take snapshots (which are stored in S3) and re-create the volume in another AZ if disaster strikes.

Volume types you'll encounter:

- gp3 / gp2 — General-purpose SSD. Default choice for most workloads.
- io2 / io1 — High-performance SSD. For demanding databases.
- st1 — Throughput-optimized HDD. For big sequential workloads like log processing.
- sc1 — Cold HDD. Cheapest, for infrequent access.

EFS — Elastic File System

EFS is fully managed NFS (Network File System). Multiple EC2 instances — even across different AZs in the same Region — can mount the same EFS file system at the same time. It scales storage automatically and you pay only for what you use.

Use EFS when you have a workload that genuinely needs shared file access across servers: think a web farm serving user uploads, or a CMS where multiple app servers must see the same content directory.

S3 — Simple Storage Service

S3 is the most famous and arguably the most important AWS service. It's object storage: every file ("object") gets a unique key, lives in a "bucket," and is retrieved via an HTTP API. It's designed for 99.999999999% ("11 nines") durability and effectively unlimited scale.

S3 storage classes — cost optimization built in

Not all data needs to be retrieved instantly. S3 offers storage classes at very different price points:

Storage class	Use case	Retrieval
S3 Standard	Frequently accessed data	Milliseconds
S3 Intelligent-Tiering	Unknown or changing access patterns — AWS moves objects between tiers automatically	Milliseconds
S3 Standard-IA	Infrequent access, kept for quick retrieval	Milliseconds (with retrieval fee)
S3 Glacier Instant Retrieval	Long-lived archives accessed quarterly	Milliseconds
S3 Glacier Flexible / Deep Archive	Cold backups and compliance archives	Minutes to hours

Lifecycle policies

You don't have to manually move data between storage classes. S3 lifecycle policies let you say things like: "After 30 days, move to Standard-IA. After 180 days, move to Glacier. After 7 years, delete." This is one of the most underused cost-saving levers on AWS.

9. Object storage deep-dive + Minio

Why object storage is its own thing

Traditional file systems organize data in a tree of folders. That model breaks down at very large scale: file system metadata becomes a bottleneck, and operations that walk directories get painfully slow.

Object storage takes a different approach. Instead of folders, every object lives in a flat namespace and is identified by a unique key. The metadata is stored alongside each object. Access happens over HTTP using a small set of verbs: PUT, GET, DELETE, LIST. This design scales horizontally to exabytes without breaking a sweat.

Because the interface is just HTTP, object storage is ideal for the web: serving images, storing user uploads, hosting static sites, backing up databases, and storing the raw input for data lakes and AI training jobs.

Minio — open-source, S3-compatible object storage

Minio is an open-source object storage server that speaks the S3 API. Anything that knows how to talk to S3 — the AWS CLI, your favorite SDK, Veeam, GitLab, MLflow — can talk to Minio with just a different endpoint URL.

Why would you self-host an object store?

- **Data sovereignty:** Keep data on hardware you control, in a country you choose, for legal or compliance reasons.
- **Cost predictability:** At very large scale, self-hosted storage can be cheaper than pay-per-GB cloud pricing.
- **Edge and air-gapped deployments:** Factories, ships, remote offices, secure facilities — places where you can't rely on public cloud connectivity.
- **Local development:** Run Minio on your laptop to mimic S3 without spending a cent.
- **Hybrid cloud:** Same API in your data center and on the public cloud means your application code doesn't need to change.

Demo preview

In tomorrow's session we'll launch an EC2 instance, install Minio on it, and treat it as a private S3-compatible store. The point isn't just to install software — it's to see that the line between "cloud" and "on-prem" can be deliberately blurred. Same protocols, different operator.

10. The Shared Responsibility Model

This is one of the most important concepts in cloud security, and it's where many beginners get caught out. The cloud is not magically secure — security is split between you and the provider.

AWS's responsibility — security "of" the cloud

AWS is responsible for protecting the infrastructure that runs all AWS services: hardware, software, networking, and physical facilities. They manage the data centers, the hypervisor, the underlying networking, and the host operating system on the bare-metal hosts.

Your responsibility — security "in" the cloud

You are responsible for everything you put on top: which operating system patches you apply to your EC2 instance, how you configure your S3 bucket permissions, how strong your passwords are, who has access to your account, and how you encrypt your data.

AWS handles	You handle
Physical security of data centers	Operating system patching on your EC2
Hardware and host OS	Application code and dependencies
Hypervisor (Nitro)	IAM users, roles, and policies
Network infrastructure	Security groups and firewall rules
Managed service software (RDS engine, S3 storage layer)	Data — encryption, classification, access
Service availability and durability SLAs	Backups of your data and your configurations

The headline beginner mistake

Making an S3 bucket public by accident and leaking sensitive data. AWS gives you all the controls to prevent this — but they're your controls, not theirs. The infamous "S3 bucket leaks" you read about in the news are almost always a customer misconfiguration, not an AWS breach.

11. Cost awareness for beginners

Bill shock is the #1 thing that scares newcomers off the cloud. Almost every beginner has a story about a \$400 bill from a forgotten resource. Here's how to avoid being that person.

The biggest cost traps

- **Leaving EC2 instances running.** Even a small instance can cost \$5–\$10 a month. A bigger one left running for a week of forgetfulness is a real expense.
- **Orphaned EBS volumes.** When you terminate an EC2 instance, the attached EBS volume sometimes survives — and you keep paying for it.
- **Elastic IPs not attached to anything.** AWS charges for unused Elastic IPs to discourage hoarding. Release them when done.
- **NAT Gateways.** Useful, but they cost about \$30/month each plus per-GB processing fees. Don't leave them up for a learning project.
- **Data transfer out.** Pulling data out of AWS to the internet is metered. A lot of beginners forget this exists.

Habits to build now

- Set a budget alert in AWS Budgets — even \$5 — before you do anything else.
- Use the AWS Free Tier when learning (12 months free t2/t3.micro, 5 GB S3, etc.).
- Stop or terminate every resource at the end of each working session.
- Tag everything with a `project` or `owner` tag so you can find what you spun up.
- Check the Cost Explorer weekly — make it a Friday afternoon habit.

Mentor's tip

Set a budget alarm BEFORE you launch your first resource. We will help you with this in the first 10 minutes of the session. Coming in cold to a \$200 surprise is preventable.

12. Pre-session checklist

Before you arrive tomorrow, please do the following. Bring questions for anything that didn't work or didn't make sense — we'll troubleshoot together in the first part of the session.

Accounts

1. Create an AWS account at <https://aws.amazon.com> — you'll need a payment method (card or supported equivalent). The Free Tier gives you 12 months of small-instance usage.
2. Immediately set up MFA on the root account, then create an IAM user for yourself. Never use the root account for day-to-day work.
3. Set a \$5 monthly budget alert in AWS Budgets. Yes, \$5 — you'll know immediately if something goes wrong.
4. (Optional) Create an Azure free account at <https://azure.microsoft.com/free> — useful for later multi-cloud sessions.

Tools to install

- AWS CLI v2 — <https://aws.amazon.com/cli/>
- Docker Desktop (Windows/macOS) or Docker Engine (Linux) — <https://www.docker.com/products/docker-desktop>
- VirtualBox — <https://www.virtualbox.org> — for the local VM exercises
- A modern terminal (Windows: Windows Terminal + WSL2; macOS/Linux: built-in)
- VS Code with the AWS Toolkit extension
- Git — <https://git-scm.com/downloads>

Quick verification commands

Run each of these in your terminal. Each one should return version information, not an error:

```
aws --version
docker --version
vboxmanage --version
git --version
```

If anything fails

Don't panic — bring screenshots of the error messages tomorrow. The first 10 minutes of the session is reserved for environment troubleshooting. You are not the only one stuck on something.

13. Glossary of terms

A quick reference of the terms we'll use repeatedly. Don't try to memorize this — refer back as needed.

Term	Meaning
AMI	Amazon Machine Image. A template (OS + pre-installed software) used to launch EC2 instances.
AZ	Availability Zone. One or more discrete data centers within an AWS Region.
Bucket	A top-level container for objects in S3 (or Minio). Globally unique name in AWS.
CDN	Content Delivery Network. Caches content close to users to reduce latency. AWS service: CloudFront.
CLI	Command-Line Interface. The text-based way to talk to AWS, often faster than the console.
Console	The AWS web UI at console.aws.amazon.com.
Container	A lightweight, portable package of an app + dependencies that shares the host OS kernel.
EBS	Elastic Block Store. Persistent block storage attached to one EC2 instance.
EC2	Elastic Compute Cloud. AWS's virtual machine service.
ECR	Elastic Container Registry. A managed Docker image registry on AWS.
ECS	Elastic Container Service. AWS-native container orchestrator.
EFS	Elastic File System. Managed shared NFS file storage.
EKS	Elastic Kubernetes Service. Managed Kubernetes on AWS.
Elasticity	The ability to scale capacity up AND down automatically with demand.
Fargate	Serverless launch type for ECS and EKS — no servers to manage.
Hypervisor	Software that creates and runs virtual machines on a physical host.
IaaS	Infrastructure as a Service. You manage OS and up.
IAM	Identity and Access Management. Who can do what in your AWS account.
Instance	A single running virtual machine on EC2.
Key pair	An SSH public/private key pair used to log in to EC2 instances.

Term	Meaning
Lambda	AWS's serverless function service (FaaS).
Minio	Open-source, S3-compatible object storage server you can self-host.
Object storage	Flat-namespace storage accessed by HTTP API (e.g., S3, Minio).
PaaS	Platform as a Service. You bring code; the provider handles the runtime.
Region	A geographic area containing multiple AWS Availability Zones.
S3	Simple Storage Service. AWS's object storage service.
SaaS	Software as a Service. End-user software delivered over the internet.
Security group	A virtual stateful firewall attached to AWS resources.
Serverless	An execution model where you don't manage servers; you pay for actual usage. Lambda, Fargate, Aurora Serverless.
Snapshot	A point-in-time backup of an EBS volume, stored in S3.
Spot	Pricing model that uses spare AWS capacity at a deep discount, with risk of interruption.
VM	Virtual Machine. An emulated computer running on a hypervisor.
VPC	Virtual Private Cloud. Your own isolated network inside AWS.

14. Further reading

None of this is required reading — but if you want to go deeper on anything in this guide, these are the resources I would point you to first.

Foundational

- AWS Cloud Practitioner Essentials (free course on AWS Skill Builder) — six hours, covers everything in this guide and more.
- The NIST Definition of Cloud Computing — SP 800-145, the document that gave us the five essential characteristics.

EC2 and compute

- AWS EC2 User Guide — long, but the canonical reference.
- Brendan Gregg, "Systems Performance" — for when you want to truly understand what's happening on your VMs.

Containers

- "Docker Deep Dive" by Nigel Poulton — friendliest introduction to containers.
- Kubernetes documentation, the "Concepts" section — read it twice.

Storage

- AWS S3 documentation, especially the sections on consistency, lifecycle, and access control.
- Minio documentation at <https://min.io/docs/minio/linux/index.html>

Cost

- AWS Well-Architected Framework — Cost Optimization Pillar. The single most useful document AWS publishes.

Watching

- AWS re:Invent keynotes and "deep dive" talks on YouTube — pick any service and search for "<service> deep dive re:Invent".

See you tomorrow

You don't need to have understood everything in this guide before the session — you need to have READ it. We'll fill in the gaps live. Come with questions. Come ready to break things in AWS. That's how you'll actually learn.